In [1]:
```
# Install once if needed:
# pip install pandas numpy gensim scikit-learn

# 1. Import libraries
import re
import numpy as np
import pandas as pd

from gensim.models import Word2Vec
from sklearn.metrics.pairwise import cosine_similarity
```

In [2]:
```
# 2. Load dataset
# Use your Twitter CSV and its tweet text column: Text
df = pd.read_csv("twitter_dataset.csv")
documents = df["Text"].dropna().astype(str).tolist()
```

In [4]:
```
# 3. Clean and tokenize documents
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"@\w+", " ", text)
    text = re.sub(r"#", " ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    return text.split()

tokenized_docs = [clean_text(doc) for doc in documents]

# Remove empty documents
valid = [(doc, tokens) for doc, tokens in zip(documents, tokenized_docs) if tokens]
documents = [item[0] for item in valid]
tokenized_docs = [item[1] for item in valid]
```

In [5]:
```
# 4. Train CBOW Word2Vec model (sg=0 means CBOW)
cbow_model = Word2Vec(
    sentences=tokenized_docs,
    vector_size=100,
    window=5,
    min_count=2,
```

```
    sg=0,
    workers=4,
    epochs=5
)
```

In [6]:
```
# 5. Represent each document by averaging its word embeddings
def document_vector(tokens):
    vectors = [
        cbow_model.wv[word]
        for word in tokens
        if word in cbow_model.wv
    ]

    if len(vectors) == 0:
        return np.zeros(100)

    return np.mean(vectors, axis=0)

doc_vectors = np.array([document_vector(doc) for doc in tokenized_docs])
```

In [8]:
```
# 6. Enter a user query
query = input("Enter keywords or a tweet: ")
query_tokens = clean_text(query)
query_vector = document_vector(query_tokens)
```

In [9]:
```
# 7. Calculate cosine similarity and recommend documents
if np.all(query_vector == 0):
    print("No query words were found in the trained vocabulary.")
else:
    similarities = cosine_similarity(
        query_vector.reshape(1, -1),
        doc_vectors
    )[0]

    top_indices = np.argsort(similarities)[::-1][:5]

    print("\nRecommended documents:")
    for i in top_indices:
```

```
        print("\nSimilarity:", round(similarities[i], 3))
        print(documents[i])
```

```
Recommended documents:

Similarity: 0.999
Short sort expect green. Report family if edge. Figure generation a.
Happy yourself range still. Also pattern thus friend religious these guy stuff. Possible admit movie.

Similarity: 0.999
Size state need chance quite. Take military once understand tend claim. Mother bad responsibility ball.
Political represent bit. Court word style.
Near doctor throughout movie white.

Similarity: 0.999
Bad different station. Beat increase certain magazine do when cell. Policy whole whom head.
Prevent particular support technology three play. Order day day by. Idea once term whatever.
Security go treat evidence claim meet. Suffer national power million bit.

Similarity: 0.999
Land will treat open reflect heart hear day. Short television as memory.
Recent four today. Top church national when team affect. Federal public compare PM Mrs.
Common bag blood financial. Matter something up.

Similarity: 0.999
Guy next among medical. Traditional all hard relationship. Modern direction left responsibility action.
They federal hand. Stand include quickly relate.
Day explain theory peace every. Which race improve property.
Near turn receive exactly. Movie outside western main majority.
```

In [ ]: